
Multilingual Conversation Translation LLM Prompt

Translate this conversation from english to *target_language*. For both the agent and customer, use the native conversation style. However, the tone and formality should match the tone and formality for each role respectively in the original English conversation. This means, for the agent, the tone should be professional and the formality should be formal, even if the customer seems to be speaking informally. respect the masculine and feminine assignments of vocabulary in the target language. The output format should be the same as the input conversation: A string with turns separated by \n exactly as done in the input transcript of the conversation, representing either the "agent" or "customer" speaking, with the actual dialogue text in the target language. Avoid using English except for the role names as explained below or placeholders indicated in the conversation. The roles in the conversation list should still be represented as "agent" or "customer". Only respond with this conversation transcript string and nothing else.
Original english conversation transcript: *conversation*
Translated conversation transcript:

Table 19: LLM prompt to translate conversations in TOD-ProcBench.

Multilingual Conversation Translations LLM Judge Prompt

Evaluate the translation quality of the translated *language* conversation with respect to the original English conversation and answer three questions with Yes or No:
English conversation: *english_conversation*
Translated conversation: *translated_conversation*
Please only provide your answers as three comma-separated Yes/No responses to the following questions and do not provide anything else in your answer:
1. Overall, did the translated conversation preserve the same meaning in each agent/customer turn as the original English conversation?
2. Overall, did the sentiment, tone, and formality of each agent/customer turn in the translated conversation match the respective agent/customer turn in the original English conversation?
3. Except for placeholder text or technical terms which do not have proper translations to the target language, is there any use of English in the translated text?

Table 20: LLM-as-a-Judge prompt to evaluate the quality of translated conversations in TOD-ProcBench.

LLM System Prompt for Task 1

You are given a part of conversation between a customer and an agent in `<conversation>` tag. In addition, you are given a set of policies in `<policies>` tag including "what_to_do" and "what_to_say" that are instructed to an agent to follow in its interaction with the customer to solve customer's provided issue.

You are also given a list of actions in `<actions>` that shows agent's next action to conduct in each point of the conversation.

Each line in `<actions>` shows one possible agent's action and the conditions necessary to select that action. The conditions are based on what customer has asked and looked for in the last turns of the conversation. In order to select the agent's next action you need to pay attention to the last agent's and customer's turns from the conversation in `<conversation>` tags.

Your task is as following:

get the two most relevant policies:

- 1) read all the policies carefully
- 2) for the given conversation select the two most matching and relevant policies based on the customer's intent from the interaction. Your selected policies should meet both of the following conditions.

a) policies state has user's provided intent or problem as an "if" condition.

b) policies include the steps that agent has followed in its interaction in the conversation to solve the problem.

get agent's next action:

- 1) read all the list of actions in `<actions>` and the conditions necessary to select each action.

- 2) read only the last agent and customer turns from `<conversation>`

- 3) find the most similar action from `<actions>` tag by comparing customer's last turn from conversation and each action's condition from `<actions>`.

Look into following examples for identifying next agent's action.

`<examples>`

`<example>`

`<conversation>`

"agent: hi! how can i help you?",

"customer: i'd like to know why i was charged twice for my subscription.",

"agent: sure, sorry about that, let me take a look",

"agent: may i have your name please?",

"customer: joseph banter",

"agent: thanks, and your account id and order id please?",

"customer: account id: `<account_id>`",

"customer: order id: `<order_id>`",

"agent: thanks! do you know your membership level?",

"customer: guest",

`<conversation>`

`<reason>` To get the agent's next action we pay attention to the last agent's and customer's turns in the `<conversation>` which is "thanks! do you know your membership level?" and "guest" where the agent has asked customer about her membership status and customer has provided it. It is exactly similar to what triggers "membership" action in `<actions>` tag that is "agent has asked customer about her membership status and customer has provided her membership level". Therefore the agent's next action is "membership".

`<reason>`

`<example>`

...

`</examples>`

Start your generation first by providing a reasoning of why you have selected the agent's next action (similar to provided examples) and the two most relevant policies in `<reason>` tags. Then return one `<output>` tag that includes THREE piece of information: the most relevant next action from `<actions>` and two most relevant policies from `<policies>` all separated by ','. The format of the output should be your provided reason in `<reason>` tags and then `<output>a,n,m</output>` without any nextline character where a is the most relevant next action from `<actions>` tag that agent should do given the last agent and customer turns in the conversation, n and m are the most relevant policies to the conversation.

Table 21: LLM system prompt for Task1.

LLM User Prompt for Task 1

List of policies: <policies> *policies* </policies>

List of next actions:

<actions>

search-policy: in the last customer turn from given <conversation> customer has asked policy related questions but agent has not searched the policy and provided info yet.

make-password: in the last customer turn from given <conversation> customer has forgotten her password and customer answered to the agent's security questions to make new password but agent has not made the password yet.

enter-details: in the last customer turn from given <conversation> customer has provided personal information such as pin, address, email, order id and etc that agent has asked. Agent next will enter those details or agent directly mentions that is going to note those details.

membership: in the last customer turn from given <conversation> customer has provided her membership status since the agent has asked about.

notify-team: in the last turn from given <conversation> agent has collected customer's issue and promised customer that is going to notify someone regarding that issue.

make-purchase: in the last customer turn from given <conversation> customer has requested agent to create a new order.

search-pricing: in the last customer turn from given <conversation> customer has requested to know about price of something and agent has not provided the price yet.

record-reason: in the last customer turn from given <conversation> customer has complained about some issue and mentioned the reason of that issue.

search-shirt: in the last customer turn from given <conversation> customer had question relevant to shirt and agent has not searched about shirt yet.

search-jacket: in the last customer turn from given <conversation> customer had question relevant to jacket and agent has not searched about jacket yet.

search-jeans: in the last customer turn from given <conversation> customer had question relevant to jeans and agent has not searched about jeans yet.

search-boots: in the last customer turn from given <conversation> customer had question relevant to boots and agent has not searched about boots yet.

search-timing: in the last customer turn from given <conversation> customer had question relevant to timing such as hours, due date and questions with 'when' and agent has not searched it to get answer yet.

promo-code: in the last customer turn from given <conversation> customer wanted to get a new promo code or agent has suggested to create promo-code to keep the customer happy and customer agreed. This action is only regarding creating new promo-code not other promo-code related requests.

log-out-in: in the last customer turn from given <conversation> customer has told about some issue and agent still has not provided logging-in and out as a solution.

shipping-status: in the last customer turn from given <conversation> customer has provided shipping status of the order to the agent.

offer-refund: in the last customer turn from given <conversation> customer has provided information such as the item brand, its cost to get a refund from agent since agent has suggested to give refund to the customer.

validate-purchase: in the last customer turn from given <conversation> customer has provided some order relevant information such as order id, account id to retrieve and validate whether there was such an order.

instructions: in the last customer turn from given <conversation> customer has talked about some issue but agent has not yet looked for some instruction and solutions such as how clearing cookies on browser.

{continued in Table 23}

Table 22: LLM user prompt (Part 1) for Task1.